# TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 1: Practical Exercises

Foreign Trade University

## Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions for the stable-matching problems (Problems 1–8, 18–22) live in `starter_code.py` at the week root: `generate_instance`, `rank`, `is_perfect_matching`, `find_rogue_couples`, `is_stable`, `print_matching`.

The 22 problems are grouped into three themes:

- **Stable matching** (Problems 1–8, 18–20, 22): the Gale–Shapley algorithm and its theoretical properties.
- **Data structures** (Problems 9–12, 16): arrays, linked lists, stacks, queues, and heaps.
- **Algorithmic building blocks** (Problems 13–17, 21): recursion, searching, sorting, divide-and-conquer multiplication, and asymptotic growth.

# 1 Stable Matching: Core Algorithm

### Problem 1 – Perfect Matching Checker

Implement `is_perfect_matching(matching, men, women)` **from scratch** (without using the helper of the same name in `starter_code.py`). A matching (a dict `man -> woman`) is *perfect* iff:

1. Every man in `men` appears exactly once as a key.
2. Every woman in `women` appears exactly once as a value, i.e. the matching is a bijection between `men` and `women`.

**Example.** With `men = ["M0","M1","M2"]`, `women = ["W0","W1","W2"]`, the matching {`"M0":"W1","M1":"W`
is perfect, but {`"M0":"W1","M1":"W1","M2":"W2"`} is not (W1 used twice, W0 missing).

### Problem 2 – Stability Checker

Implement `find_rogue_couples(matching, men_prefs, women_prefs)` **from scratch**. A pair $(m, w)$ is a *rogue couple* (blocking pair) for `matching` if:

- $m$ and $w$ are *not* matched to each other, **and**
- $m$ prefers $w$ to his current partner, **and**
- $w$ prefers $m$ to her current partner.

Return a list of all such $(m, w)$ pairs. A matching is *stable* iff this list is empty.

**Example.** Two men $\{M0, M1\}$, two women $\{W0, W1\}$, where both men prefer W0 and both women prefer M0 (i.e. everyone agrees W0 ¿ W1 and M0 ¿ M1). The matching $\{M0{:}W0, M1{:}W1\}$ is stable. The matching $\{M0{:}W1, M1{:}W0\}$ is *unstable*: $(M0, W0)$ is a rogue couple, since $M0$ prefers $W0$ to $W1$ and $W0$ prefers $M0$ to $M1$.

### Problem 3 – Gale–Shapley Algorithm (Men Propose)

Implement `gale_shapley_men_propose(men_prefs, women_prefs)`:

```
Initialize all m in M and w in W to free.
While some man m is free and hasn't proposed to every woman:
    w = first woman on m's list to whom m has not yet proposed
    if w is free:
        (m, w) become engaged
    else if w prefers m to her current partner m':
        m' becomes free; (m, w) become engaged
    else:
        w rejects m
```

Return the resulting matching as a dict `man -> woman`. Verify on a random instance (via `generate_instance`) that the result is a perfect matching (Problem 1) and stable (Problem 2).

### Problem 4 – Gale–Shapley Algorithm (Women Propose)

By symmetry with Problem 3, implement `gale_shapley_women_propose(men_prefs, women_prefs)`, where the roles are swapped: women propose to men, and men hold the best offer received so far. Return the matching in the *same orientation* as Problem 3 (`man -> woman`), so the two results are directly comparable. *Hint*: you can implement this by calling `gale_shapley_men_propose` with the roles of `men_prefs` and `women_prefs` swapped, then flipping the resulting dict.

### Problem 5 – Brute-Force Enumeration of All Stable Matchings

For small $n$, implement `all_stable_matchings(men, women, men_prefs, women_prefs)` that returns a list of *all* stable perfect matchings, by:

1. Generating every possible perfect matching (every permutation of `women` assigned to `men`, i.e. $n!$ candidates), using `itertools.permutations`.
2. Keeping only those that are stable (Problem 2 / `is_stable`).

This is exponential and only intended for small $n$ (say $n \leq 6$); it exists so we can empirically verify properties of Gale–Shapley in Problems 6–8.

### Problem 6 – How Many Stable Matchings Are There?

Using `all_stable_matchings` from Problem 5, implement `average_num_stable_matchings(n, num_trials, seed)`:

1. Generate `num_trials` random instances of size $n$ (with `generate_instance(n, seed=seed + trial_index)` or similar, so each trial is different but reproducible).
2. Compute the number of stable matchings for each instance.
3. Return the average over all trials, as a float.

In `__main__`, print this average for $n = 1, \ldots, 5$ with `num_trials=20` and observe how quickly the count grows with $n$.

### Problem 7 – Verifying the Men-Optimality Theorem

**Theorem (Kleinberg & Tardos, Thm 1.7).** In the matching produced by the men-proposing Gale–Shapley algorithm, every man receives the *best valid partner* he could have in *any* stable matching (he is "man-optimal").

A woman $w$ is a *valid partner* of man $m$ if there exists *some* stable matching in which $m$ and $w$ are matched.

Implement:

- `valid_partners(person, all_stable, role)` – returns the set of valid partners of `person`, where `role` is `"man"` or `"woman"`, using the list of all stable matchings from Problem 5.
- `verify_men_optimality(men, men_prefs, all_stable, men_optimal)` – checks, for every man $m$, that `men_optimal[m]` is $m$'s *most-preferred* valid partner.

Test by generating a random instance, computing `all_stable_matchings` (Problem 5) and `gale_shapley_men_prop` (Problem 3), and confirming `verify_men_optimality` returns `True`.

### Problem 8 – Verifying the Women-Pessimality Theorem

**Theorem (Kleinberg & Tardos, Thm 1.8).** In the same matching, every woman is matched with her *least-preferred* valid partner (she is "woman-pessimal").

Using `valid_partners` from Problem 7, implement `verify_women_pessimality(women, women_prefs, all_stable, men_optimal)` that checks, for every woman $w$, that her partner in `men_optimal` is her *least*-preferred valid partner. Test the same way as Problem 7.

## 2 Data Structures

### Problem 9 – Array vs. Linked List: Counting Operations

Implement two simple sequence data structures, each instrumented with an `ops` counter that counts "basic steps" (element shifts for the array, pointer hops for the linked list):

- `ArraySeq.prepend(x)`: insert `x` at index 0. Every existing element must be shifted right by one slot → costs `len(self.data)` ops.
- `ArraySeq.get(i)`: direct indexing → costs 1 op.
- `LinkedSeq.prepend(x)`: create a new head node → costs 1 op.
- `LinkedSeq.get(i)`: walk `i` pointers from the head → costs $(i + 1)$ ops.

Then implement `total_prepend_ops(seq_class, n)` that creates a fresh `seq_class()`, prepends $n$ items $(0, 1, \ldots, n-1$ in that order), and returns the final value of `seq.ops` (the total cost of all prepends).

**What you should observe.** For `ArraySeq`, the total is $0 + 1 + \cdots + (n-1) = \binom{n}{2} = \Theta(n^2)$. For `LinkedSeq`, the total is exactly $n = \Theta(n)$. This is a direct, measurable illustration of the array-vs-linked-list trade-off from the lecture notes.

### Problem 10 – Stack: Balanced Brackets Checker

Implement a simple `Stack` class with `push`, `pop`, `peek`, and `is_empty`, backed by a Python list. Then implement `is_balanced(expr)` that returns `True` iff every `(`, `[`, { in `expr` is closed by the matching `)`, `]`, } in the correct order (other characters are ignored).

**Examples.** `"(a[b]{c})"` → `True`. `"(a[b)]"` → `False` (wrong order). `"((a)"` → `False` (unclosed).

### Problem 11 – Queue: Round-Robin CPU Scheduling

Implement a `Queue` class backed by a Python list with `enqueue`, `dequeue`, and `is_empty` (FIFO order).

Then implement `round_robin(tasks, quantum)`, where `tasks` is a list of `(name, burst_time)` pairs. Simulate round-robin scheduling: repeatedly dequeue a task, run it for up to `quantum` time units, and if it still has remaining time, enqueue it again at the back. Return the order in which tasks *finish* (a list of names, in finishing order).

**Example.** `tasks = [("A",5), ("B",3), ("C",8)]`, `quantum=4`. Trace: A runs 4 (1 left), B runs 3 (done), C runs 4 (4 left), A runs 1 (done), C runs 4 (done). Finishing order: `["B", "A", "C"]`.

### Problem 12 – Singly Linked List

Implement a `LinkedList` class with:

- `append(x)`: add `x` at the end – $O(n)$ without a tail pointer.
- `prepend(x)`: add `x` at the front – $O(1)$.
- `delete(x)`: remove the first node with value `x` (no-op if not found).

- `find(x)`: return `True` iff `x` is in the list.
- `to_list()`: return a Python list of the values, in order.
- `reverse()`: reverse the list *in place* (re-link nodes – do not just reverse `to_list()`).

### Problem 16 – Binary Min-Heap and Heapsort

Implement an array-based binary `MinHeap` with:

- `push(x)`: insert `x`, then "sift up" to restore the heap property.
- `pop()`: remove and return the minimum element, moving the last element to the root and "sifting down" to restore the heap property. Raise `IndexError` if empty.
- `__len__`: number of elements.

Then implement `heapsort(arr)` that returns a new sorted list by pushing all elements onto a `MinHeap` and popping them off in order. This gives an $O(n \log n)$ sort.

# 3 Algorithmic Building Blocks

### Problem 13 – Binary Search (Iterative and Recursive)

Implement two versions of binary search over a sorted list `arr`:

- `binary_search_iterative(arr, target)`: returns the index of `target` in `arr`, or $-1$ if not present. Use a `while` loop.
- `binary_search_recursive(arr, target)`: same behaviour, implemented recursively (a helper with `lo`/`hi` bounds is fine).

Also implement `recursion_depth(n)`, returning the number of recursive calls `binary_search_recursive` would make in the *worst case* on a sorted array of length $n$, i.e. $\lceil \log_2(n+1) \rceil$ – the number of halvings until the search range is empty.

### Problem 14 – Insertion Sort with Comparison Counter

Implement `insertion_sort(arr)` that returns a *new* sorted list (do not mutate the input) using the standard insertion-sort algorithm, together with the total number of element *comparisons* performed, as a tuple `(sorted_list, comparisons)`.

**Verify empirically** that:

- On an already-sorted list of length $n$, insertion sort makes exactly $n-1$ comparisons (best case, $O(n)$).
- On a reverse-sorted list of length $n$, it makes exactly $\frac{n(n-1)}{2}$ comparisons (worst case, $O(n^2)$).

### Problem 15 – Karatsuba Multiplication

Implement `karatsuba(x, y)`, multiplying two non-negative integers using the Karatsuba divide-and-conquer algorithm: given $x, y$ with $m$ digits each, split $x = a \cdot 10^{m/2} + b$ and $y = c \cdot 10^{m/2} + d$. Then

$$ac = a \cdot c, \qquad bd = b \cdot d, \qquad ad\_plus\_bc = (a+b)(c+d) - ac - bd,$$

$$x \cdot y = ac \cdot 10^{m} + ad\_plus\_bc \cdot 10^{m/2} + bd$$

using *three* recursive multiplications instead of four. Base case: if $x < 10$ or $y < 10$, return $x \cdot y$ directly.

Also implement `naive_multiply_op_count(x, y)` that returns the number of single-digit multiplications a grade-school algorithm would need, roughly `len(str(x)) * len(str(y))`.

### Problem 17 – Asymptotic Growth Classification

You are given a dictionary mapping function names to callables of $n$:

```
FUNCTIONS = {
    "constant":     lambda n: 1,
    "logarithmic":  lambda n: math.log2(n),
    "sqrt":         lambda n: math.sqrt(n),
    "linear":       lambda n: n,
    "linearithmic": lambda n: n * math.log2(n),
    "quadratic":    lambda n: n ** 2,
```

```
    "cubic":        lambda n: n ** 3,
    "exponential":  lambda n: 2 ** n,
}
```

Implement `order_by_growth(functions, n)` that returns the list of function names sorted by *increasing* value at the given $n$ (i.e. from slowest- to fastest-growing at that $n$).

Then implement `dominates(f, g, n_values)` that returns `True` iff, for every $n$ in `n_values` (a list of increasing positive integers), $f(n) \leq g(n)$ – a simple empirical proxy for "$f$ is $O(g)$".

### Problem 21 – Recursion: Naive vs. Memoized Fibonacci

Implement `fib_naive(n)` returning `(value, call_count)` where `value` is the $n$-th Fibonacci number ($F(0) = 0$, $F(1) = 1$) and `call_count` is the total number of recursive calls made (including the initial call).

Then implement `fib_memo(n)` returning `(value, call_count)` using memoization (a dict cache), where `call_count` should be much smaller – $O(n)$ instead of exponential.

**What you should observe.** For $n = 20$: `fib_naive` makes 21891 calls, while `fib_memo` makes only 39.

# 4 Stable Matching: Extensions & Trace Practice

### Problem 18 – Empirical Running Time of Gale–Shapley

Implement `count_proposals(men_prefs, women_prefs)`, a variant of the men-proposing Gale–Shapley algorithm (Problem 3) that returns the *total* number of proposals made (instead of the final matching).

The Kleinberg–Tardos textbook proves this is at most $n^2$ (Section 1.1's analysis). Implement `average_proposals(n, num_trials, seed)` returning the average number of proposals over `num_trials` random instances of size $n$, and verify it never exceeds $n^2$.

### Problem 19 – Hospital–Residents Problem (Many-to-One Stable Matching)

Generalize Gale–Shapley to the Hospital–Residents problem: each hospital $h$ has a capacity `capacity[h]` (it can accept up to that many residents). Residents propose to hospitals; a hospital holds the best `capacity[h]` residents it has seen so far (by its preference list) and rejects the rest.

Implement `hospital_resident_matching(residents, hospitals, resident_prefs, hospital_prefs, capacity)`:

- `residents`, `hospitals`: lists of names.
- `resident_prefs[r]`: hospitals in order of preference (best first).
- `hospital_prefs[h]`: residents in order of preference (best first).
- `capacity[h]`: max number of residents hospital $h$ can take.

Return a dict `hospital -> list of assigned residents`, where every hospital's list has length $\leq$ `capacity[h]`, and the assignment is stable: no resident $r$ and hospital $h$ both prefer each other over $r$'s current assignment / one of $h$'s current residents.

### Problem 20 – Stable Matching with Incomplete Preference Lists

In practice, not everyone is acceptable to everyone else. Generalize Gale–Shapley to *incomplete* preference lists: `men_prefs[m]` and `women_prefs[w]` list only the *acceptable* partners, best first (a person may have an empty list).

Run the men-proposing algorithm: each man proposes down his list. If he runs out of acceptable women, he remains permanently single. Women only accept proposals from men on their own list.

Implement `gale_shapley_incomplete(men, women, men_prefs, women_prefs)` returning a dict `man -> woman` containing *only* the matched pairs (unmatched men/women are simply absent from the dict).

### Problem 22 – Trace-Based Verification on a Textbook Example

Consider the classic $3 \times 3$ instance (Kleinberg & Tardos, Chapter 1):

| Men's preferences (best first) | Women's preferences (best first) |
|---|---|
| $X$: A, B, C | $A$: Y, X, Z |
| $Y$: B, A, C | $B$: X, Y, Z |
| $Z$: A, B, C | $C$: X, Y, Z |

Implement `build_instance()` returning `(men, women, men_prefs, women_prefs)` for this instance, where `men = ["X","Y","Z"]` and `women = ["A","B","C"]`.

Using `gale_shapley_men_propose` from Problem 3, the men-proposing algorithm on this instance produces the matching $\{X{:}A, Y{:}B, Z{:}C\}$. You can verify this by hand-tracing the algorithm:

| Step | Proposal | Outcome | Engaged pairs after step |
|---|---|---|---|
| 1 | $X \to A$ | $A$ free, accepts | $(X, A)$ |
| 2 | $Y \to B$ | $B$ free, accepts | $(X, A), (Y, B)$ |
| 3 | $Z \to A$ | $A$ prefers $X$ to $Z$ (A's list: Y,X,Z) – $Z$ rejected | $(X, A), (Y, B)$ |
| 4 | $Z \to B$ | $B$ prefers $Y$ to $Z$ (B's list: X,Y,Z) – $Z$ rejected | $(X, A), (Y, B)$ |
| 5 | $Z \to C$ | $C$ free, accepts | $(X, A), (Y, B), (Z, C)$ |

Note that $Z$, the only man rejected along the way, ends up with his *last* choice $C$ – while $A$ and $B$ (the women he was rejected by) each end up with their *first*-choice man among those who proposed to them. This single trace illustrates both the men-optimal and women-pessimal theorems from Problems 7–8.

Implement `verify_textbook_example()` that builds the instance, runs Gale–Shapley, and returns `True` iff the result equals `{"X":"A","Y":"B","Z":"C"}`.